



UNIVERSIDAD AGRARIA DEL ECUADOR

FACULTAD DE CIENCIAS AGRARIAS

CARRERA DE INGENIERÍA EN COMPUTACIÓN

ASIGNATURA:

LENGUAJE DE PROGRAMACION 2

GRUPO 2:

PEDRO SEGUNDO ABARCA ORRALA

HENRY ALEXANDER DELACRUZ CASTILLO

GENESSIS MILENA LOPEZ MENDOZA

FRANCISCO ANDRES PIEDRA ORTEGA

JULEXI TATIANA QUINDE EUGENIO

CURSO:

3SA

DOCENTE:

ING. BRYAN ORLANDO VELEZ SAN MARTIN

FECHA:

21 DE JULIO DE 2026

Índice de contenidos

INTRODUCCIÓN.....	4
JUSTIFICACIÓN	5
OBJETIVOS.....	6
Objetivo General	6
Objetivos Específicos	6
ALCANCE.....	7
La Propuesta Incluye:	7
La propuesta no incluye:	8
MARCO TEÓRICO.....	10
Programación Orientada A Objetos (Poo)	10
Clases y objetos: definición, atributos y métodos	10
Encapsulamiento y modificadores de acceso.....	10
Herencia simple y múltiple.....	10
Polimorfismo y sobrecarga de métodos.....	11
Abstracción e interfaces	12
Estructuras De Datos Lineales	12
Listas enlazadas simples: nodos, inserción y eliminación	12
Listas doblemente enlazadas y circulares	12
Pilas (Stack): operaciones push, pop y peek	13
Colas (Queue): operaciones enqueue y dequeue	13
Aplicaciones prácticas de pilas y colas	13
Algoritmos De Ordenamiento	15
Algoritmos básicos: Burbuja, Selección e Inserción	15
Algoritmos avanzados: Merge Sort y Quick Sort	15
Algoritmos de búsqueda	16
Búsqueda lineal	16
Búsqueda binaria.....	16
METODOLOGÍA DE DESARROLLO	17
Análisis de requerimientos.....	17

Diseño orientado a objetos	17
Implementación de estructuras de datos	18
Desarrollo de la base de datos	18
Desarrollo de la interfaz gráfica.....	19
Pruebas del sistema	20
Herramientas utilizadas.....	21
<i>DISEÑO PRELIMINAR DEL SISTEMA</i>	<i>22</i>
<i>IMPLEMENTACIÓN</i>	<i>24</i>
<i>PRUEBAS REALIZADAS</i>	<i>26</i>
Evidencia.....	27
<i>CONCLUSIONES Y RECOMENDACIONES.....</i>	<i>33</i>
Conclusiones:.....	33
Recomendaciones:.....	34
<i>BIBLIOGRAFÍA</i>	<i>35</i>
<i>ANEXOS.....</i>	<i>36</i>

INTRODUCCIÓN

El desarrollo de sistemas informáticos se ha convertido en un elemento fundamental para mejorar la organización y eficiencia de los procesos administrativos, especialmente en el sector de la salud, donde el manejo adecuado de la información influye directamente en la calidad de la atención brindada a los pacientes. En muchas clínicas, la administración de citas, historiales clínicos y datos de pacientes todavía presenta dificultades relacionadas con la organización, el acceso rápido a la información y el seguimiento de los procesos, lo que puede generar retrasos o inconsistencias durante la atención.

Con el propósito de aportar una solución a esta problemática, el presente proyecto desarrolla un sistema de gestión de citas médicas para una clínica privada, diseñado para administrar de forma organizada el registro de pacientes, médicos, citas e historiales clínicos. El sistema fue modelado mediante diagramas UML y un modelo entidad-relación que permitieron definir la estructura de la aplicación antes de su implementación, facilitando la relación entre los diferentes componentes y garantizando un diseño coherente con las necesidades planteadas.

La implementación del sistema se realizó en Python utilizando los principios de la Programación Orientada a Objetos, lo que permitió representar mediante clases las principales entidades del sistema, como Persona, Paciente, Médico, Cita e Historial Clínico. Asimismo, se incorporó una base de datos SQLite para el almacenamiento de la información, una interfaz gráfica desarrollada con Tkinter para facilitar la interacción con el usuario y diversas estructuras de datos, como colas, pilas y listas, que optimizan procesos como la atención de pacientes, la gestión del historial clínico y la administración de las citas médicas.

En conjunto, este proyecto demuestra cómo la integración entre el análisis del sistema, el diseño mediante diagramas, las estructuras de datos y la programación orientada a objetos permite desarrollar una aplicación funcional, organizada y escalable, capaz de responder a necesidades reales dentro del entorno de una clínica, al mismo tiempo que fortalece los conocimientos adquiridos durante la asignatura de Lenguaje de Programación II.

JUSTIFICACIÓN

El desarrollo de un sistema de gestión de citas médicas para una clínica privada responde a la necesidad de optimizar la administración de pacientes, médicos, citas e historiales clínicos mediante una herramienta informática que permita organizar la información de forma eficiente y segura. La automatización de estos procesos contribuye a disminuir errores en el registro de datos, reducir los tiempos de atención y facilitar el acceso oportuno a la información, aspectos fundamentales para mejorar la calidad del servicio en las instituciones de salud (Hernández & Baquero, 2025).

Desde el punto de vista académico, este proyecto constituye una oportunidad para aplicar los conocimientos adquiridos en la asignatura de Lenguaje de Programación, integrando los principios de la Programación Orientada a Objetos, como clases, objetos, encapsulamiento, herencia y polimorfismo, con el propósito de modelar correctamente las entidades que conforman el sistema. Asimismo, permite emplear estructuras de datos lineales y algoritmos de búsqueda y ordenamiento para resolver problemas propios de un entorno real, fortaleciendo las competencias necesarias para el desarrollo de aplicaciones de software (G, 2024b; Valdivia et al., 2022).

Desde una perspectiva práctica, el sistema permite gestionar de manera organizada el registro de pacientes y médicos, programar citas según la especialidad correspondiente y controlar el orden de atención mediante una cola (FIFO). Asimismo, el historial clínico reciente de cada paciente se administra mediante una pila (LIFO), facilitando el acceso a la información más reciente. En cuanto a la navegación cronológica de las citas, se plantea como una mejora para futuras versiones la implementación de una lista doblemente enlazada, la cual permitirá recorrer la agenda médica en ambos sentidos y optimizar la gestión de las citas conforme el sistema continúe evolucionando (Bel, 2020).

En conjunto, el proyecto demuestra la importancia de aplicar las estructuras de datos, los algoritmos y la Programación Orientada a Objetos en el desarrollo de soluciones informáticas que respondan a necesidades reales, promoviendo aplicaciones más organizadas, eficientes, escalables y fáciles de mantener (Hernández & Baquero, 2025).

OBJETIVOS

Objetivo General

Desarrollar un sistema de gestión de citas médicas para una clínica privada mediante Python, aplicando los principios de la Programación Orientada a Objetos, estructuras de datos, una base de datos SQLite y una interfaz gráfica, con el propósito de administrar de forma organizada la información de pacientes, médicos, citas e historiales clínicos, optimizando los procesos de atención y consulta.

Objetivos Específicos

- Diseñar e implementar las clases que conforman el sistema, estableciendo las relaciones entre pacientes, médicos, citas e historiales clínicos mediante los principios de la Programación Orientada a Objetos para representar adecuadamente el funcionamiento de la aplicación.
- Integrar una base de datos SQLite que permita almacenar y gestionar de forma segura la información de pacientes, médicos, citas e historiales clínicos, facilitando las operaciones de registro, consulta y administración de los datos.
- Implementar estructuras de datos como colas, pilas y listas para optimizar el manejo de la sala de espera, el historial clínico y la organización de las citas médicas, de acuerdo con la lógica de funcionamiento del sistema.
- Incorporar algoritmos de búsqueda y organización que permitan localizar y gestionar eficientemente la información registrada, mejorando el rendimiento de las consultas realizadas por el usuario.
- Desarrollar una interfaz gráfica intuitiva mediante Tkinter que facilite la interacción entre el usuario y el sistema, permitiendo ejecutar las principales funciones relacionadas con la gestión de pacientes, médicos, citas e historiales clínicos.

ALCANCE

El alcance del presente proyecto define de manera explícita los límites conceptuales, funcionales y técnicos del Sistema de Gestión de Citas Médicas destinado a la clínica privada, garantizando que el diseño y la estructuración de datos respondan con precisión a las necesidades identificadas. Como señala la literatura técnica, "el alcance del software describe las funciones, características y restricciones operativas que se entregarán a los usuarios finales, estableciendo una línea base clara para el diseño arquitectónico" (Pressman & Maxim, 2024, p. 145). Por lo tanto, se delimitan rigurosamente los componentes que la propuesta incluye y aquellos que quedan explícitamente excluidos del desarrollo inicial.

La Propuesta Incluye:

1. **Gestión de Entidades Principales (POO):** Modelado e implementación de las clases fundamentales del sistema: Paciente, Médico, Cita Médica e Historial Clínico, parametrizando sus respectivos atributos esenciales y métodos de encapsulamiento.
 - **Relación con diagramas:** En el Diagrama de Clases UML (Ilustración 1), este componente se consolida mediante la jerarquía de herencia desde la superclase Persona (que provee los atributos protegidos de nombre y cédula) hacia las clases hijas Paciente y Medico con visibilidad privada (-). En el Modelo Entidad-Relación (Ilustración 2), este modelado se traduce en la persistencia de datos mediante las entidades físicas independientes Paciente, Medico, Cita e Historial_Clinico junto con sus respectivos atributos definidos en elipses.
2. **Módulo de Sala de Espera Operativa:** Aplicación de una estructura de datos tipo Cola (FIFO) para simular y administrar de forma ordenada el flujo de pacientes que esperan atención médica en el día corriente.
 - **Relación con diagramas:** En el Diagrama UML (Ilustración 1), la operatividad de esta estructura dinámica en memoria se refleja en la firma del método atenderPaciente() de la clase Medico, encargado de procesar y desencolar al siguiente turno. En el Modelo ER (Ilustración 2), este flujo se respalda mediante el atributo Estado de la entidad Cita, que

permite filtrar y listar de manera secuencial a aquellos pacientes que se encuentran con estado "En Espera".

3. **Módulo de Consultas Históricas:** Integración de una estructura de datos tipo Pila (LIFO) orientada al almacenamiento dinámico y visualización prioritaria del historial reciente de atenciones médicas de cada paciente.

➤ **Relación con diagramas:** En el Diagrama UML (Ilustración 1), se representa a través de la relación de Asociación directa de 1 a muchos (*) entre Paciente e HistorialClinico, accionada mediante el método de consulta consultarHistorial(). En el Modelo ER (Ilustración 2), esta correspondencia se modela en la relación Tiene con cardinalidad de (1,1) a (1,N), lo que permite almacenar los atributos de Diagnostico y Tratamiento vinculados a un paciente para ser apilados y mostrados cronológicamente de forma inversa en la aplicación.

4. **Componente de Búsqueda y Ordenamiento:** Inclusión de algoritmos avanzados de búsqueda para la localización exacta de pacientes y médicos, junto con algoritmos de ordenamiento para clasificar la información según criterios clave como la especialidad médica, fecha y bloques de hora.

➤ **Relación con diagramas:** En el Diagrama UML (Ilustración 1), el uso de estos algoritmos es indispensable para optimizar y asegurar el rendimiento interno de métodos críticos como consultarHistorial() y consultarAgenda(). En el Modelo ER (Ilustración 2), los algoritmos operan de manera directa sobre los atributos de búsqueda definidos en las elipses de las entidades, específicamente sobre la Cedula (en Paciente) para búsquedas exactas, y sobre los campos Especialidad (en Medico), Fecha y Hora (en Cita) para los criterios de ordenación lineal y logarítmica.

La propuesta no incluye:

1. **Integración Externa de Pasarelas de Pago:** No se contempla la vinculación con servicios bancarios ni plataformas transaccionales para el cobro o facturación de las citas médicas de la clínica.

2. **Despliegue en Entornos de Producción Cloud:** El software está limitado a una ejecución y simulación en entorno local, omitiendo configuraciones de servidores en la nube, infraestructura web compleja o protocolos de balanceo de carga.
3. **Interoperabilidad con Sistemas de Salud Externos:** No se incluye la sincronización con bases de datos gubernamentales, farmacias externas ni sistemas integrados de salud a nivel nacional.
4. **Módulos Proyectados para Futuras Implementaciones:** Aunque el Módulo de Agenda Cronológica fue excluido del alcance actual de la propuesta, el sistema queda diseñado para permitir su integración en el futuro mediante una lista doblemente enlazada para la navegación bidireccional de citas médicas. Esta escalabilidad está respaldada técnicamente en el Diagrama UML actual, a través del método `consultarAgenda()`, y en el Modelo E-R, aprovechando las relaciones de cardinalidad de uno a muchos (1:N) ya establecidas entre las entidades Paciente, Medico y Cita.

La importancia de establecer estas barreras radica en que "la delimitación formal de lo que un sistema de información no hará constituye una salvaguarda crítica para mitigar la corrupción del alcance y el desbordamiento de recursos en la ingeniería de software" (Sommerville, 2022, p. 88). De este modo, el proyecto se concentra exclusivamente en optimizar la lógica de negocio y las estructuras de datos requeridas para la clínica privada.

MARCO TEÓRICO

Programación Orientada A Objetos (Poo)

Clases y objetos: definición, atributos y métodos

La Programación Orientada a Objetos es un paradigma que organiza el software en unidades llamadas objetos, cada uno de ellos como instancia de una clase que define sus atributos (información que describe su estado) y sus métodos (acciones u operaciones que puede realizar) (Hernández & Baquero, 2025). Este enfoque promueve la reutilización del código, el modularidad y una relación más natural entre un problema del mundo real y su representación en software). En el sistema desarrollado, la clase Paciente ilustra muy bien este concepto: establece los atributos nombre, cédula, edad y teléfono, y ofrece métodos de acceso (`get_nombre()`, `get_cedula()`, etc.) junto con un método de comportamiento (`mostrar_datos()`) que presenta la información del objeto. El método `_init_` funciona como constructor: se ejecuta de forma automática al crear una instancia y se encarga de inicializar su estado (G, 2024a).

Encapsulamiento y modificadores de acceso

El encapsulamiento consiste en limitar el acceso directo a los datos internos de un objeto, dejando visibles únicamente los métodos necesarios para interactuar con ellos. En Python, este mecanismo no se implementa con palabras reservadas como `private` o `public` (al estilo de Java o C++), sino que se sigue una convención de nombres: cuando un atributo lleva doble guion bajo (`_atributo`), se activa el name mangling, lo que dificulta su acceso directo desde fuera de la clase. Este enfoque se utiliza en la clase Paciente, donde los cuatro atributos se definen como privados (`self._nombre`, `self._cedula`, `self._edad`, `self._telefono`) y solo pueden consultarse mediante los métodos `get_*`(), lo que protege la integridad de la información del paciente y evita modificaciones no controladas desde otras partes del sistema (G, 2024a).

Herencia simple y múltiple

La herencia permite crear una nueva clase, conocida como subclase o clase hija, a partir de una clase ya existente, llamada superclase o clase padre, heredando sus atributos y métodos, y

ofreciendo la posibilidad de ampliarlos o sobrescribirlos. En Python, la herencia se declara pasando la clase padre como parámetro en la definición: `class Hija(Padre):`. A diferencia de otros lenguajes como Java, Python soporta la herencia múltiple, lo que significa que una clase puede heredar de más de una clase padre al mismo tiempo, algo muy útil para modelar entidades que comparten características provenientes de diferentes orígenes (G, 2024a; Valdivia et al., 2022).

Polimorfismo y sobrecarga de métodos

El polimorfismo es la capacidad de que objetos de diferentes clases respondan de manera distinta a un mismo mensaje o llamada de método. En Python, esto se ve sobre todo a través de la sobreescritura de métodos, donde una subclase redefine el comportamiento de un método heredado, y también, de forma más flexible que en lenguajes con tipado estático, mediante el uso de argumentos por defecto o variables (*args, **kwargs) que imitan la sobrecarga de métodos, ya que Python no permite definir de forma nativa varios métodos con el mismo nombre, pero distinta firma (Bahit, 2022)

Abstracción e interfaces

La abstracción se trata de enfocarse solo en los aspectos esenciales de una entidad que son relevantes para el problema que se quiere resolver, dejando de lado los detalles de implementación que no son necesarios para quien utiliza el objeto (Hernández & Baquero, 2025). En Python, este concepto se implementa formalmente con el módulo abc (Abstract Base Classes), que permite crear clases abstractas e interfaces con métodos que las subclasses deben obligatoriamente desarrollar. En un sistema de citas médicas, este enfoque puede servir para definir una interfaz común llamada Persona, de la cual hereden Paciente y Médico, asegurando que ambas clases cuenten con métodos como mostrar_datos().

Estructuras De Datos Lineales

Listas enlazadas simples: nodos, inserción y eliminación

Una lista enlazada simple es una estructura dinámica formada por nodos, donde cada nodo contiene un dato y una referencia (puntero) al siguiente nodo. A diferencia de una lista con acceso indexado, no necesita memoria contigua, lo que permite insertar y eliminar elementos en tiempo constante cuando se conoce la posición del nodo, sin tener que desplazar el resto de los elementos (G, 2024b; Bel, 2020).

Listas doblemente enlazadas y circulares

Una lista doblemente enlazada extiende el concepto anterior agregando a cada nodo una referencia adicional al nodo anterior, lo que permite recorrer la estructura en ambos sentidos (Bel, 2020). Esta característica es la base teórica para la navegación anterior/siguiente de la agenda de citas del sistema, ya que permite avanzar o retroceder entre citas consecutivas sin recorrer la lista completa desde el inicio. Una variante es la lista circular, en la que el último nodo enlaza nuevamente con el primero, útil para representar ciclos (por ejemplo, turnos rotativos de atención).

Pilas (Stack): operaciones push, pop y peek

La pila es una estructura de datos lineal que sigue el principio LIFO (Last In, First Out), donde el último elemento insertado es el primero en salir (Bel, 2020). Sus operaciones fundamentales son: push, que inserta un elemento en la cima de la pila; pop, que extrae y elimina el elemento de la cima; y peek (o top), que permite consultar el elemento superior sin eliminarlo. En Python, una lista nativa puede funcionar como pila utilizando `append()` para realizar la operación push y `pop()` (sin índice) para ejecutar pop, ya que ambas se llevan a cabo sobre el final de la lista con complejidad $O(1)$ (G, 2024a). Esta estructura constituye la base teórica para el historial reciente de atenciones de cada paciente. Sistema: Cada nueva atención se apila mediante push sobre las anteriores, y al consultar el historial se accede primero a la atención más reciente.

Colas (Queue): operaciones enqueue y dequeue

La cola es una estructura de datos lineal que sigue el principio FIFO (First In, First Out), donde el primer elemento insertado es el primero en salir (Bel, 2020). Sus operaciones fundamentales son: enqueue, que inserta un elemento al final de la cola, y dequeue, que extrae y elimina el elemento situado al frente de la misma. Python ofrece en su biblioteca estándar el tipo `collections.deque`, una estructura optimizada que permite inserciones y eliminaciones en $O(1)$ tanto al inicio como al final, lo que la hace preferible a una lista nativa para implementar colas. Esta es la estructura utilizada en el sistema para representar la sala de espera de pacientes del día. El método `append()` cumple la función de enqueue (ingreso a la sala de espera) y `popleft()` la de dequeue (llamado del paciente para su atención), respetando el orden de llegada (FIFO) requerido por el contexto del proyecto.

Aplicaciones prácticas de pilas y colas

Las pilas se emplean habitualmente en la gestión de historiales de navegación, funciones de deshacer/rehacer y control de llamadas recursivas; en este proyecto, en el historial clínico reciente

por paciente. Las colas se emplean en sistemas de impresión, procesamiento de tareas y, en este caso, en la administración del orden de atención de la sala de espera, garantizando equidad en el turno de los pacientes (Bel, 2020).

Algoritmos De Ordenamiento

Algoritmos básicos: Burbuja, Selección e Inserción

Ordenamiento burbuja (Bubble Sort) recorre repetidamente la colección comparando elementos adyacentes e intercambiándolos si están en el orden incorrecto, de modo que en cada pasada el elemento de mayor valor "burbujea" hacia el final (Bel, 2020). Complejidad $O(n^2)$ en el peor caso.

El ordenamiento por selección, conocido como Selection Sort, consiste en que en cada iteración se elige el elemento mínimo o máximo) que queda por ordenar y se coloca en su posición final dentro del arreglo. Este método tiene una complejidad de $O(n^2)$.

El ordenamiento por inserción, también conocido como Insertion Sort, arma la colección ordenada paso a paso, añadiendo cada nuevo elemento en el lugar exacto en relación con los que ya están acomodados. Aunque su complejidad es $O(n^2)$, resulta bastante eficiente cuando se trata de colecciones pequeñas o que ya están casi ordenadas, lo que lo hace una opción práctica en esos casos.

Algoritmos avanzados: Merge Sort y Quick Sort

Merge Sort (ordenamiento por mezcla) utiliza la estrategia de divide y vencerás, separando la colección en mitades de forma recursiva hasta obtener elementos individuales, para luego combinarlos o “mezclarlos” en orden. Ofrece una complejidad de $O(n \log n)$ en todos los casos.

Quick Sort, o también llamado ordenamiento rápido, se basa en el método de divide y vencerás: elige un elemento pivote y reorganiza la colección para que los valores menores queden a la izquierda y los mayores a la derecha, repitiendo el proceso de forma recursiva. Su complejidad promedio es $O(n \log n)$, aunque en el peor de los casos puede llegar a $O(n^2)$ (Bel, 2020).

En el sistema, estos algoritmos son aplicables para ordenar, por ejemplo, la lista de pacientes por edad, la agenda de citas por hora o el historial clínico por fecha.

Algoritmos de búsqueda

Búsqueda lineal

Este método consiste en recorrer la colección paso a paso, revisando cada elemento hasta dar con el valor que se busca o llegar al final de la estructura. No es necesario que los datos estén ordenados y su complejidad es $O(n)$ (Bel, 2020). Un ejemplo claro de su aplicación sería encontrar a un paciente en la fila de espera utilizando su número de cédula.

Búsqueda binaria

Este método necesita que la colección esté ordenada de antemano. Consiste en comparar el elemento que se quiere encontrar con el valor central y, en cada paso, descartar la mitad en la que no puede estar, reduciendo así el espacio de búsqueda de forma logarítmica hasta dar con él. Su complejidad es $O(\log n)$, lo que lo hace mucho más eficiente que la búsqueda lineal cuando se manejan grandes cantidades de datos (Bel, 2020). Un ejemplo de uso sería localizar rápidamente una cita en una agenda ya organizada por fecha y hora.

METODOLOGÍA DE DESARROLLO

Para el desarrollo del **Sistema de Gestión de Citas Médicas** se empleó una metodología de desarrollo incremental, debido a que permitió construir el sistema por módulos, implementando y probando cada funcionalidad antes de integrar la siguiente. Este enfoque facilitó detectar errores de manera temprana y verificar el correcto funcionamiento de cada componente del sistema.

El proceso de desarrollo se organizó en las siguientes etapas:

Análisis de requerimientos

En la primera etapa se identificaron las necesidades de la clínica privada, definiendo las funcionalidades principales del sistema, entre ellas:

- Registro de pacientes.
- Registro de médicos.
- Agendamiento de citas médicas.
- Administración del historial clínico.
- Gestión de la sala de espera.
- Consulta de información almacenada.

Además, se determinaron las entidades principales del sistema y las relaciones existentes entre ellas para elaborar el modelo e-r y el diagrama UML.

Diseño orientado a objetos

Posteriormente se diseñó la estructura del sistema utilizando los principios de la Programación Orientada a Objetos.

Se implementaron las siguientes clases del dominio:

- Persona
- Paciente
- Médico
- Cita

- HistorialClinico

La clase **Persona** funciona como clase base o Padre, mientras que **Paciente** y **Médico** heredan sus atributos y métodos, aplicando el principio de herencia. Asimismo, los atributos fueron encapsulados mediante modificadores privados y métodos Getter y Setter para proteger la información del sistema. Esto puede observarse en la definición de las clases del código fuente.

Implementación de estructuras de datos

Para representar situaciones reales del contexto médico se emplearon diferentes estructuras de datos:

- **Cola (FIFO)** mediante la estructura **deque** de Python para representar la sala de espera de pacientes.
- **Pila (LIFO)** mediante listas para almacenar el historial reciente de atenciones.
- **Lista** para gestionar las citas médicas registradas.

Estas estructuras permiten organizar la información respetando el comportamiento requerido por cada proceso del sistema.

Desarrollo de la base de datos

La persistencia de la información se implementó utilizando **SQLite**, mediante la biblioteca **sqlite3**.

Se creó una base de datos denominada **clinica.db**, en la cual se almacenan los registros correspondientes a:

- Pacientes
- Médicos
- Citas médicas
- Historial clínico

Además, se implementó la creación automática de las tablas y la inserción inicial de datos de prueba para verificar el funcionamiento del sistema.

Desarrollo de la interfaz gráfica

La interfaz gráfica del sistema fue desarrollada utilizando la biblioteca **Tkinter**, incluida en Python, con el objetivo de crear un prototipo visual que represente la estructura inicial del Sistema de Gestión de Citas Médicas.

En esta etapa del proyecto se diseñó la ventana principal incorporando elementos básicos de una interfaz gráfica, como el título principal, subtítulo, etiquetas informativas, botones, colores, tipos de letra y la organización de los componentes mediante contenedores. Estos elementos permiten presentar una distribución clara y amigable para el usuario.

La interfaz incluye botones que representan las principales opciones del sistema, tales como:

- Registrar paciente.
- Consultar historial.
- Registrar médico.
- Consultar agenda.
- Agendar cita.
- Cancelar o reprogramar cita.
- Agregar registro clínico.
- Mostrar historial.

En la versión actual del proyecto, el botón **Registrar paciente** fue implementado de manera funcional. Al seleccionarlo, el sistema abre una ventana de registro donde el usuario puede ingresar el nombre, la cédula, la edad y el teléfono del paciente. La información ingresada es validada antes de almacenarse en la base de datos SQLite, verificando que todos los campos estén completos, que la edad corresponda a un valor numérico y que la cédula no se encuentre registrada previamente. Una vez completado el proceso, el sistema informa al usuario el resultado mediante mensajes de confirmación o de error.

Los demás botones de la interfaz continúan funcionando como un prototipo visual y muestran ventanas informativas (MessageBox) para representar las funcionalidades que serán implementadas en las siguientes etapas del desarrollo, tales como el registro de médicos, la consulta de historiales y el agendamiento de citas.

Pruebas del sistema

Durante el desarrollo del sistema se realizaron pruebas básicas con el propósito de verificar el correcto funcionamiento de los componentes implementados hasta la etapa actual del proyecto.

En primer lugar, se comprobó la creación de la base de datos **SQLite**, verificando que el archivo **clinica.db** se generara correctamente y que las tablas correspondientes a pacientes, médicos, citas e historial clínico fueran creadas sin errores. Asimismo, se validó la inserción de los registros de prueba definidos en el código y la consulta de la información almacenada.

También se realizaron pruebas sobre las estructuras de datos implementadas, comprobando el funcionamiento de la cola para representar la sala de espera de pacientes, la pila para el historial clínico reciente y las listas utilizadas para el manejo de las citas médicas. Estas pruebas permitieron verificar que las operaciones básicas de inserción, eliminación y recorrido se ejecutaran correctamente.

Respecto a la interfaz gráfica, se comprobó el correcto funcionamiento de la ventana principal desarrollada con Tkinter, verificando la visualización del título, subtítulo, etiquetas, botones y demás elementos gráficos del sistema. Asimismo, se validó el funcionamiento del botón **Registrar paciente**, el cual abre una ventana de registro, permite ingresar y validar los datos del paciente y almacena correctamente la información en la base de datos SQLite. Por otra parte, los demás botones continúan formando parte del prototipo de la interfaz, mostrando ventanas informativas mediante tkinter.messagebox para representar las funcionalidades que serán implementadas en las siguientes etapas del desarrollo.

Herramientas utilizadas

Las principales herramientas empleadas durante el desarrollo fueron:

Herramienta	Uso
Python 3	Desarrollo del sistema
Visual Studio Code	Edición del código fuente
Tkinter	Desarrollo de la interfaz gráfica
SQLite	Base de datos relacional
DB Browser for SQLite	Administración y visualización de la base de datos
GitHub	Control de versiones y trabajo colaborativo

DISEÑO PRELIMINAR DEL SISTEMA

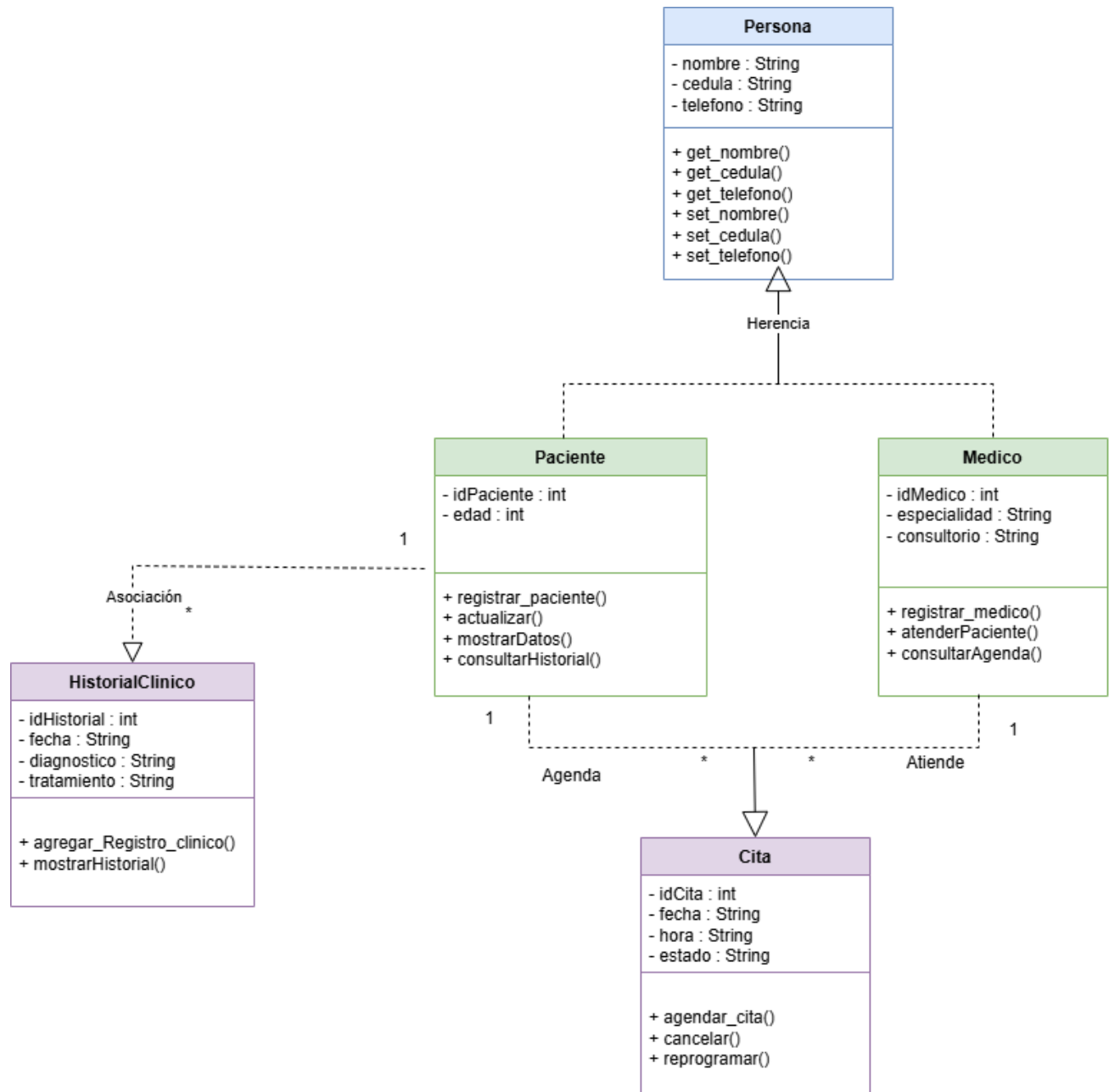


Ilustración 1: Diagrama de clases UML

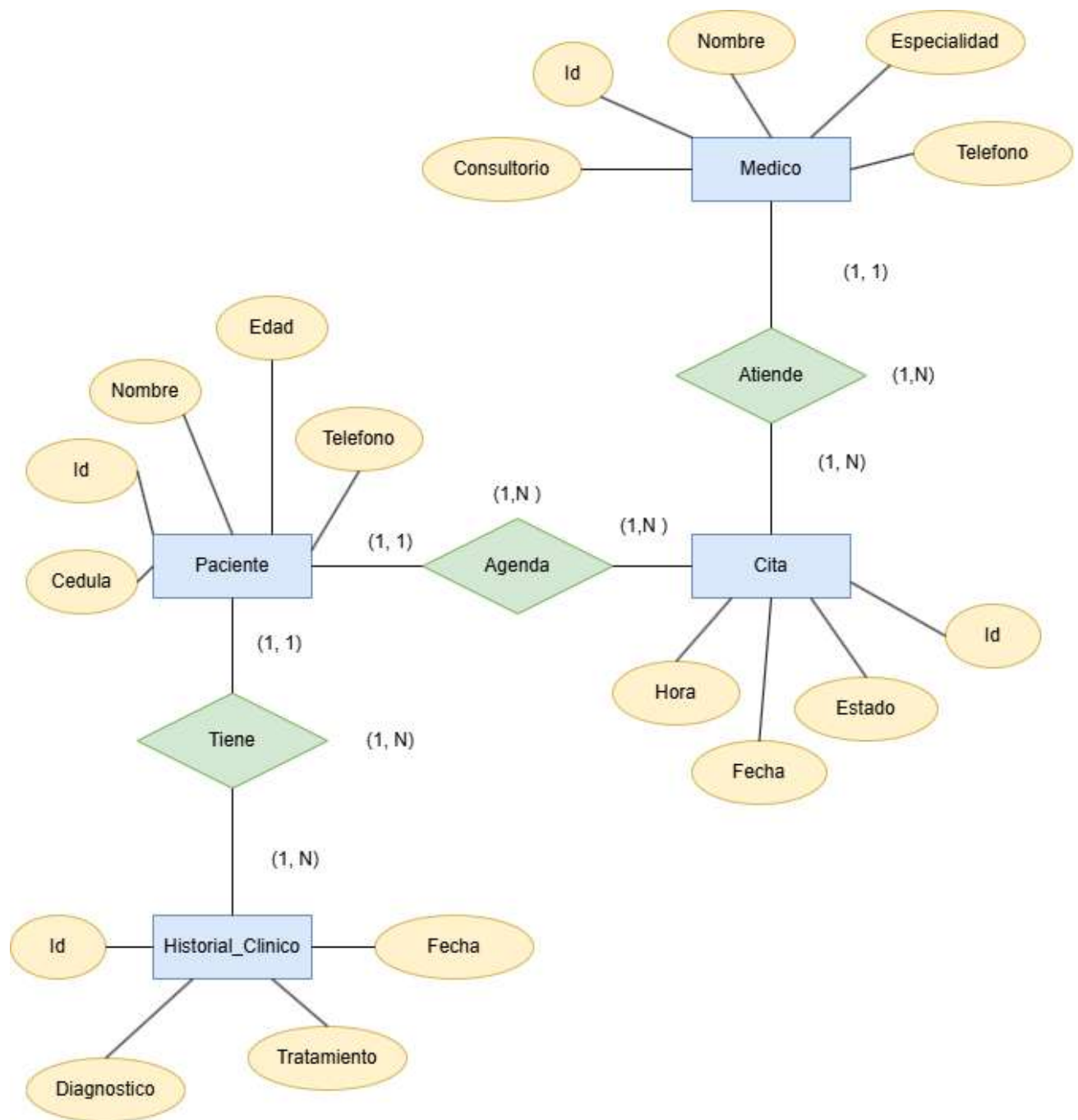


Ilustración 2: Modelo entidad-relación preliminar de la base de datos

IMPLEMENTACIÓN

La implementación del sistema de gestión de citas médicas se realizó utilizando el lenguaje de programación Python, siguiendo la estructura previamente definida en los diagramas UML y en el modelo entidad-relación. Durante esta etapa se trasladó el diseño conceptual a una aplicación funcional, procurando que cada uno de los componentes desarrollados mantuviera una relación directa con las entidades y funcionalidades planteadas en el análisis del sistema.

Como base del desarrollo se aplicó el paradigma de la Programación Orientada a Objetos. Para ello se implementó la clase Persona, de la cual heredan las clases Paciente y Medico, permitiendo reutilizar atributos comunes como nombre, cédula y teléfono mediante el mecanismo de herencia. Además, se implementaron las clases Cita e HistorialClinico, encargadas de representar la información correspondiente a las citas médicas y al historial de atención de cada paciente. Cada clase incorpora constructores, métodos de acceso (getters), métodos modificadores (setters) y atributos encapsulados, favoreciendo una mejor organización del código y protegiendo la integridad de la información.

En cuanto a las estructuras de datos, el sistema incorpora una cola (FIFO) mediante la estructura `deque`, utilizada para simular el orden de atención de los pacientes en la sala de espera, garantizando que el primer paciente registrado sea el primero en ser atendido. Asimismo, se implementó una pila (LIFO) para representar el historial clínico reciente, permitiendo acceder primero al registro más reciente del paciente. Para la gestión de las citas también se empleó una lista que permite recorrer la información almacenada y administrar los registros existentes de manera sencilla.

El sistema también incorpora operaciones básicas de búsqueda sobre la información registrada, permitiendo localizar pacientes mediante recorridos secuenciales dentro de las estructuras implementadas. Estas operaciones facilitan la consulta de los datos y constituyen la base para futuras mejoras relacionadas con algoritmos de búsqueda y ordenamiento más eficientes.

Para almacenar la información generada por la aplicación se utilizó una base de datos SQLite, donde se crearon las tablas correspondientes a Paciente, Medico, Cita e Historial_Clinico, respetando las relaciones definidas durante el diseño del sistema. Además de la creación de las tablas, se implementó la inserción de registros iniciales y consultas básicas que permiten verificar el correcto funcionamiento de la aplicación y la persistencia de los datos.

Finalmente, se desarrolló una interfaz gráfica utilizando la biblioteca Tkinter, la cual proporciona un entorno amigable para la interacción con el usuario. La interfaz incluye botones para registrar pacientes y médicos, consultar historiales, gestionar la agenda médica, programar o cancelar citas y visualizar las diferentes funcionalidades implementadas. Aunque algunas opciones funcionan actualmente como demostración mediante ventanas informativas, la estructura desarrollada permite ampliar fácilmente el sistema e incorporar nuevas funcionalidades en futuras versiones.

En conjunto, la implementación permitió integrar los principios de la Programación Orientada a Objetos, las estructuras de datos y la interfaz gráfica en una aplicación organizada y funcional, demostrando la aplicación práctica de los conocimientos adquiridos durante la asignatura y sentando las bases para el desarrollo de un sistema de gestión de citas médicas más completo.

PRUEBAS REALIZADAS

Con el propósito de verificar el correcto funcionamiento de los componentes implementados en el Sistema de Gestión de Citas Médicas, se realizaron diversas pruebas funcionales durante el desarrollo del proyecto. Estas pruebas permitieron comprobar el comportamiento de la base de datos, las estructuras de datos y la interfaz gráfica desarrollada con Tkinter. Asimismo, se verificó que cada uno de los elementos implementados respondiera de acuerdo con el alcance actual del proyecto, documentando los resultados obtenidos mediante evidencias que respaldan el funcionamiento del sistema.

Nº	Prueba realizada	Resultado esperado	Resultado obtenido	Evidencia
1	Creación automática de la base de datos	Se crea el archivo <code>clinica.db</code> con sus tablas	La base de datos se creó correctamente	Figura 1
2	Inserción de pacientes	Los pacientes se almacenan en SQLite	Los registros fueron insertados correctamente	Figura 2
3	Consulta de pacientes	Mostrar todos los pacientes registrados	Se visualizaron correctamente	Figura 3
4	Funcionamiento de la cola	El primer paciente es atendido primero (FIFO)	La cola funcionó correctamente	Figura 4
5	Funcionamiento de la pila	Se elimina primero el último diagnóstico agregado	La pila funcionó correctamente	Figura 5
6	Interfaz gráfica	La ventana principal carga correctamente	La interfaz se mostró sin errores	Figura 6
7	Botones de la interfaz	Mostrar una ventana MessageBox	Todos los botones mostraron el mensaje correspondiente	Figura 7
8	Registro de paciente desde la interfaz gráfica.	El sistema debe permitir ingresar los datos del paciente y almacenarlos en la base de datos SQLite.	El paciente fue registrado correctamente y la información quedó almacenada en la tabla <code>paciente</code> .	Figuras 8, 9 y 10.

Evidencia

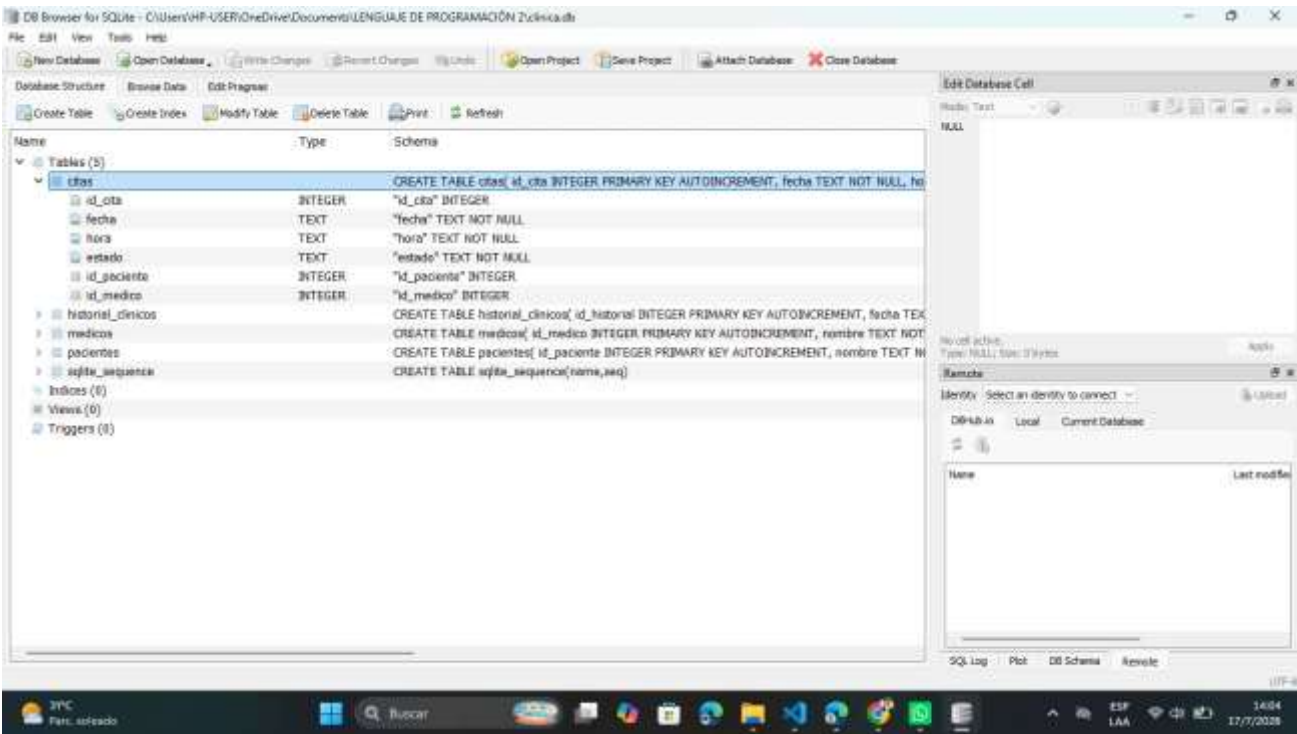


Figura 1. Base de datos *clinica.db* visualizada en DB Browser mostrando las tablas creadas.

Table: pacientes					
	<u>id_paciente</u>	nombre	cedula	edad	telefono
	Filter	Filter	Filter	Filter	Filter
1	1	Pedro Vera	0954789652	21	0984697459
2	2	María López	0912345678	30	0914795672
3	3	Carlos Pizarro	0985867412	20	0974965178

Figura 2. Registros de pacientes almacenados en la tabla Paciente de la base de datos.

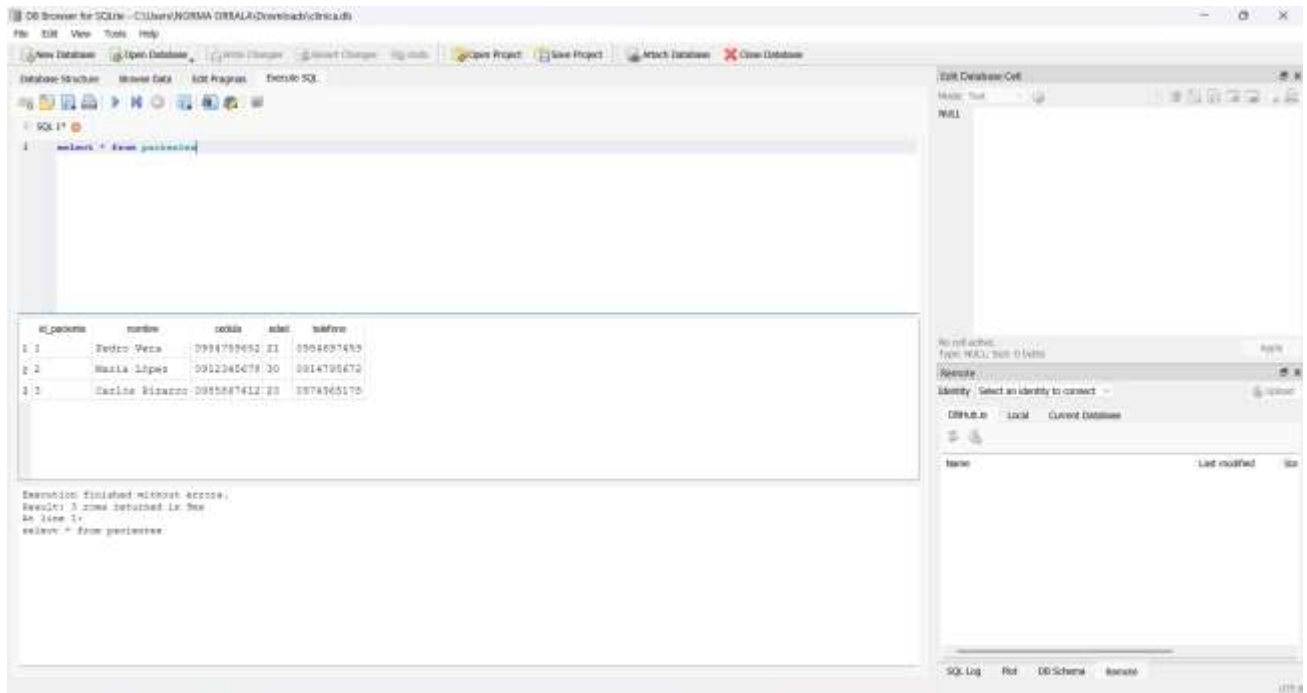


Figura 3. Consulta de los pacientes registrados ejecutada desde el programa.

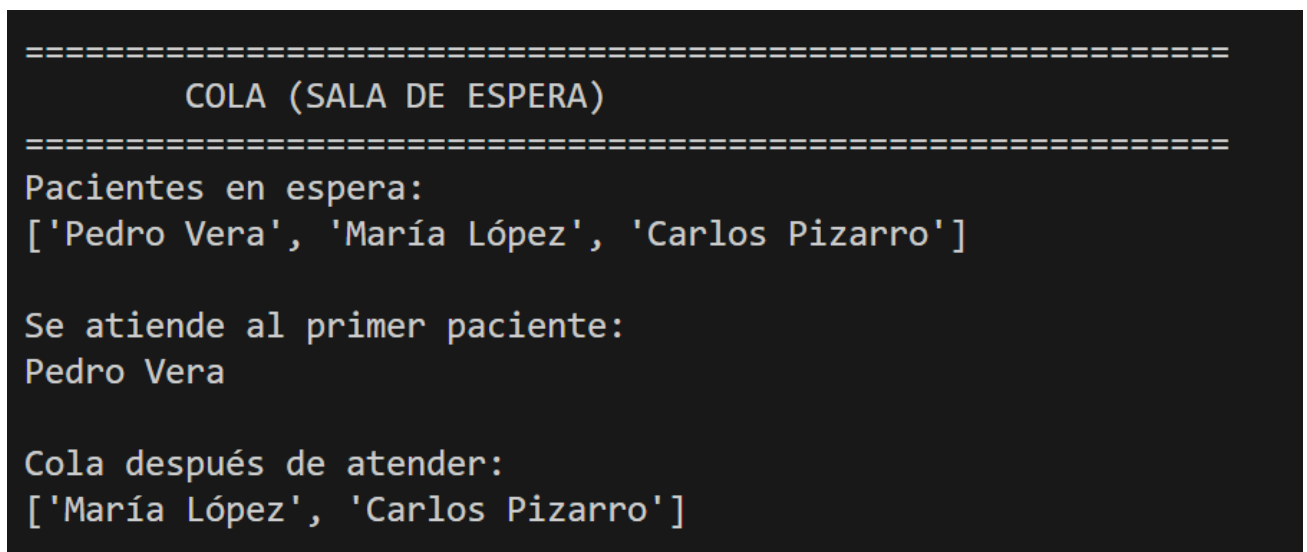


Figura 4. Prueba de funcionamiento de la cola (FIFO) para la sala de espera.

```
=====
PILA (HISTORIAL CLÍNICO)
=====
Historial almacenado:
['Gripe', 'Migraña', 'Alergia']

Última atención registrada:
Alergia

Se elimina la última atención

Historial actualizado
['Gripe', 'Migraña']
```

Figura 5. Prueba de funcionamiento de la pila (LIFO) para el historial clínico.

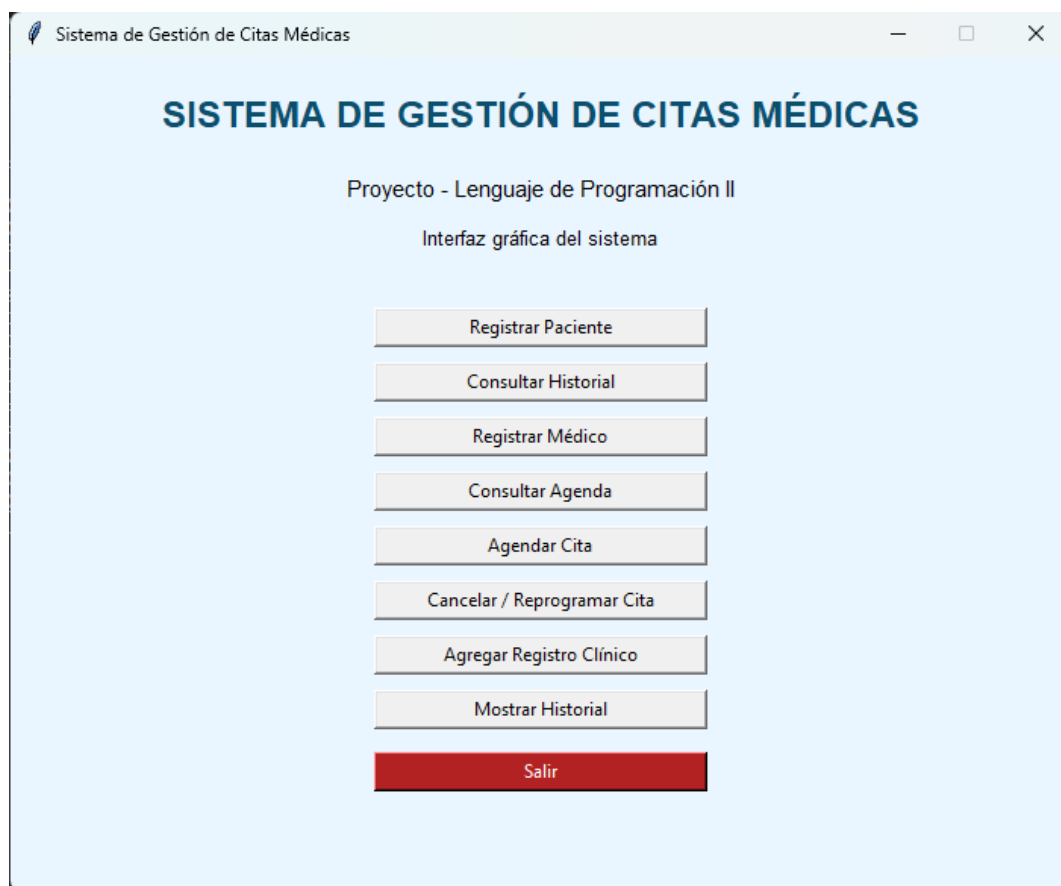


Figura 6. Ventana principal de la interfaz gráfica desarrollada con Tkinter.

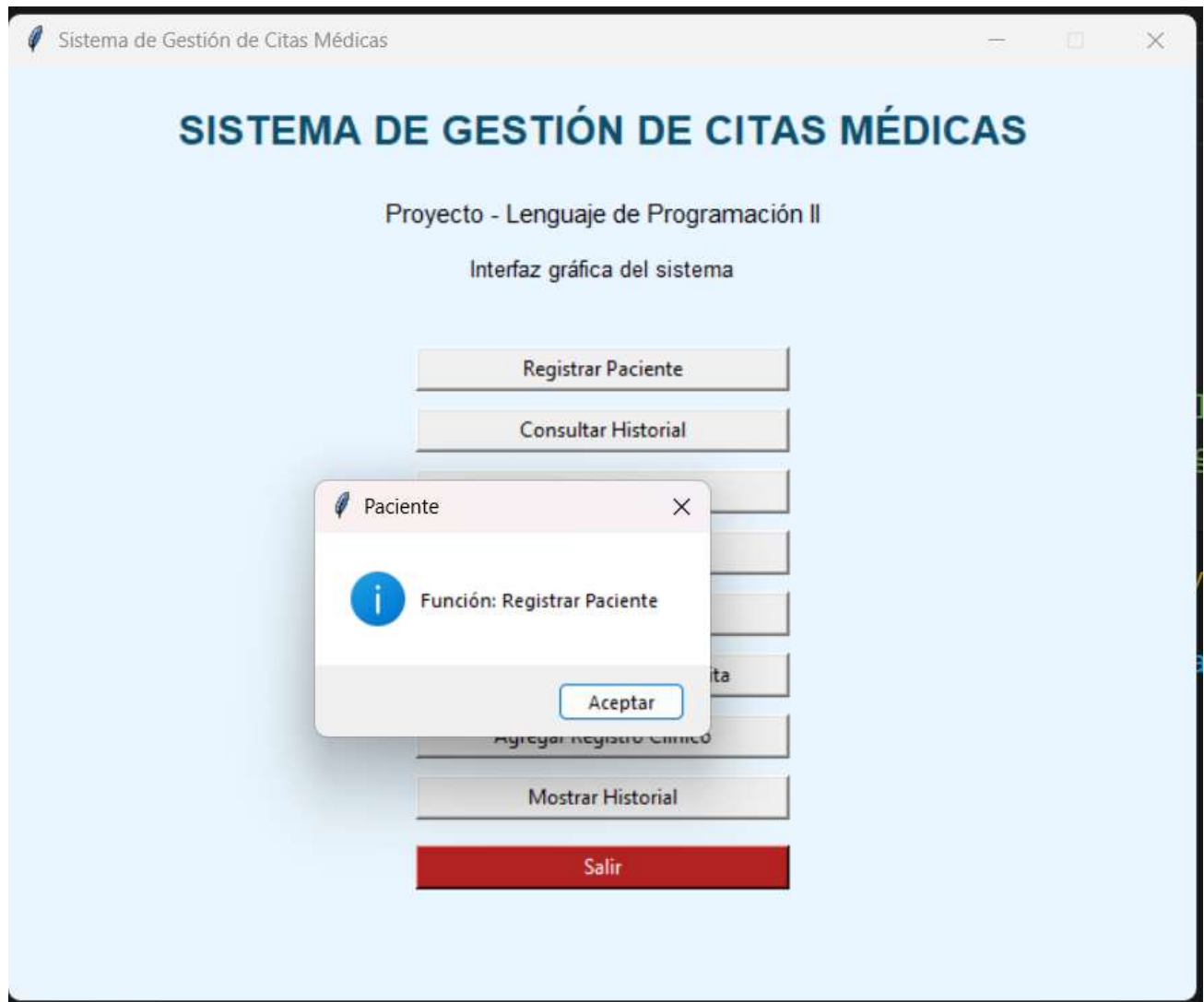


Figura 7. Ventana emergente (MessageBox) mostrada al seleccionar una opción de la interfaz.

The image shows a window titled "Registrar Paciente" with a light blue background. It contains four input fields labeled "Nombre:", "Cédula:", "Edad:", and "Teléfono:". Below these fields is a button labeled "Guardar".

Figura 8. Ventana de registro de pacientes de la interfaz gráfica.

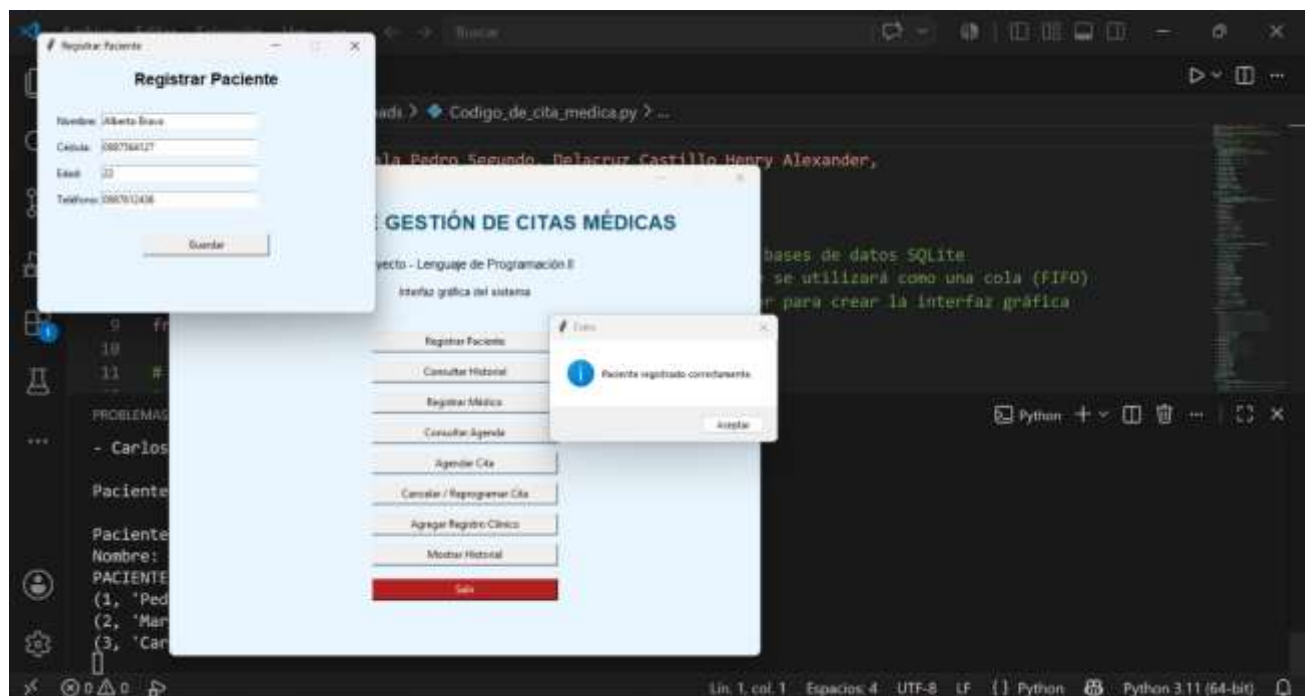


Figura 9. Registro exitoso de un paciente mediante la interfaz gráfica.

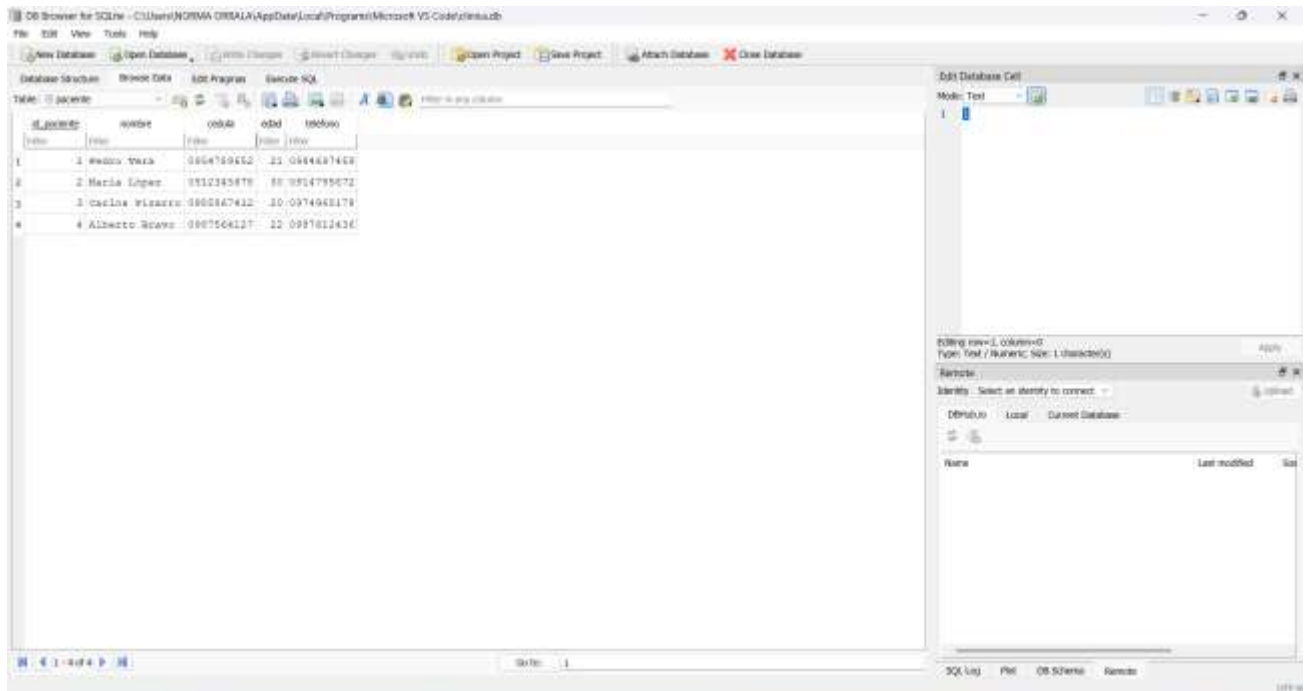


Figura 10. Nuevo paciente almacenado en la tabla Paciente de la base de datos SQLite.

Las pruebas realizadas permitieron verificar el correcto funcionamiento de los componentes desarrollados hasta la etapa actual del proyecto. Se comprobó la creación y administración de la base de datos SQLite, el comportamiento de las estructuras de datos implementadas y la correcta visualización de la interfaz gráfica elaborada con Tkinter. Asimismo, se confirmó que los botones responden adecuadamente mediante ventanas emergentes (MessageBox), cumpliendo con el propósito de representar las funcionalidades previstas para futuras versiones del sistema. En conjunto, las pruebas evidencian que el proyecto cuenta con una base sólida para continuar con la implementación de las funcionalidades que serán integradas en las siguientes etapas de desarrollo.

CONCLUSIONES Y RECOMENDACIONES

Conclusiones:

- **Viabilidad del Paradigma Orientado a Objetos:** Se demostró que la aplicación de los principios fundamentales de la Programación Orientada a Objetos (POO), tales como la herencia simple a partir de la superclase Persona hacia Paciente y Médico, junto con el encapsulamiento estricto de atributos mediante el uso de modificadores de acceso privados y métodos getters y setters, garantiza un diseño de software altamente cohesionado, con bajo acoplamiento y con una estructura modular idónea para la representación fidedigna de las reglas de negocio de una entidad de salud.
- **Eficiencia Operativa mediante Estructuras Dinámicas:** La integración estratégica de estructuras de datos lineales resolvió de manera óptima los flujos operativos de la clínica. El uso de colas (FIFO) a través del módulo collections.deque de Python aseguró un ordenamiento equitativo en la sala de espera; el almacenamiento en estructuras de pilas (LIFO) permitió una recuperación cronológicamente inversa y eficiente del historial clínico reciente; y el modelado conceptual de listas doblemente enlazadas sentó las bases para una navegación bidireccional ágil a través de la agenda cronológica de citas.
- **Garantía de Persistencia y Consistencia de Datos:** La implementación de la base de datos relacional en SQLite, estructurada en estricta conformidad con el Modelo Entidad-Relación preliminar, demostró ser una solución técnica robusta, ligera y eficiente para entornos locales. Las pruebas de persistencia evidenciaron que las operaciones de creación automatizada de tablas (pacientes, medicos, citas e historial_clinicos) e inserción de registros iniciales se ejecutan sin inconsistencias, asegurando la integridad referencial a través de llaves foráneas.
- **Efectividad del Prototipado de Interfaz de Usuario:** El desarrollo de la interfaz gráfica de usuario mediante la biblioteca Tkinter cumplió exitosamente con el objetivo de proveer un esquema visual intuitivo y centralizado. Aunque los componentes lógicos actúan en esta fase como un prototipo funcional demostrativo y el manejo de eventos se gestiona temporalmente a través de ventanas emergentes informativas (tkinter.messagebox), la arquitectura desacoplada de la interfaz facilita una vinculación directa e inmediata con los métodos de manipulación de datos en fases posteriores del desarrollo.

Recomendaciones:

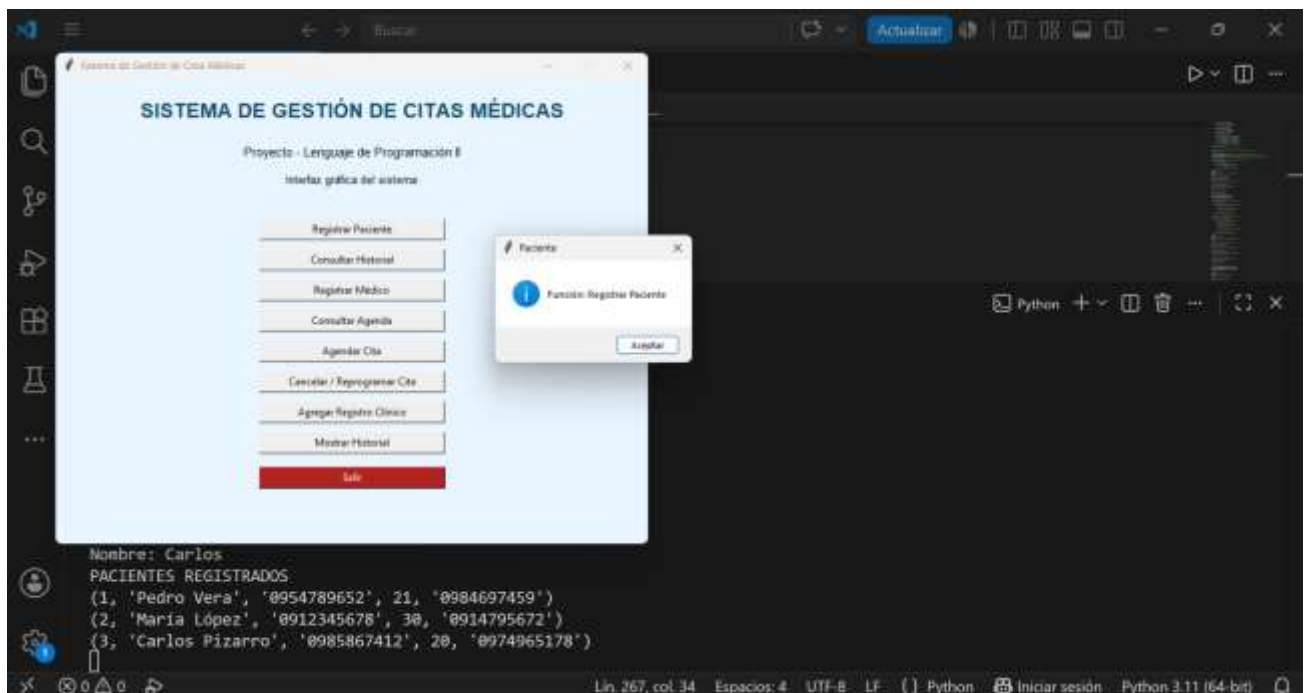
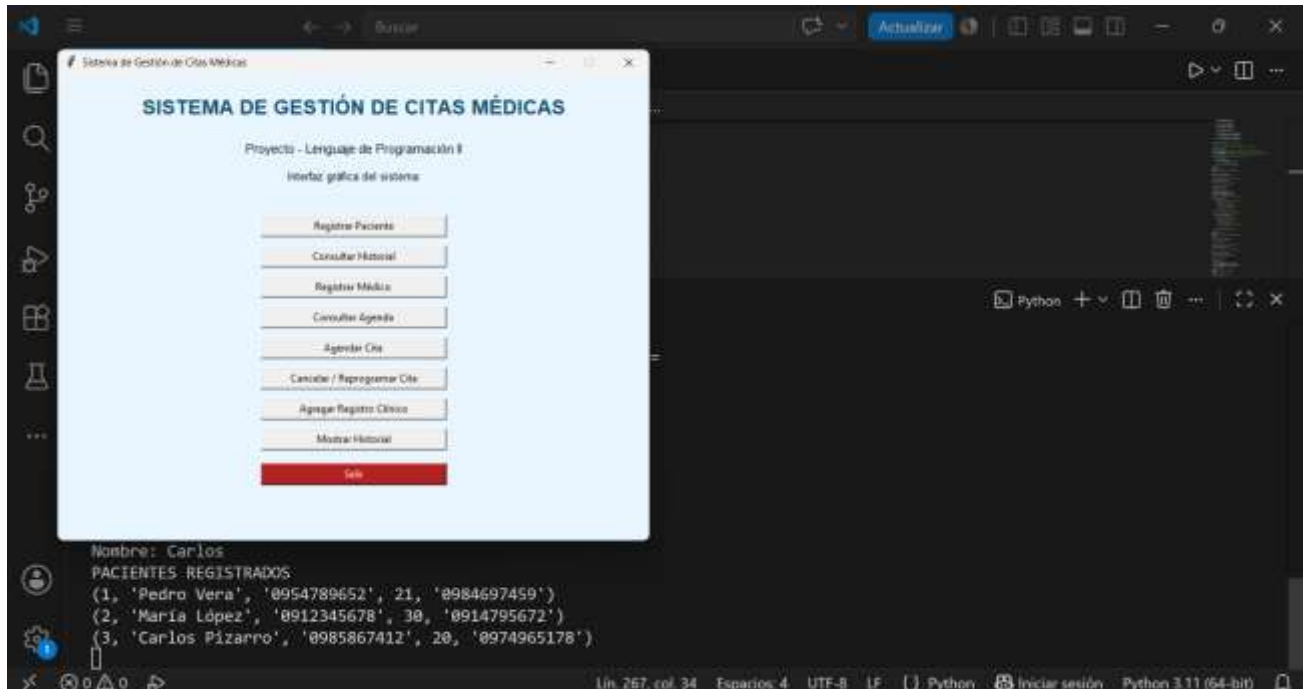
- ***Evolución e Integración de la Capa Lógica con la Interfaz Gráfica:*** Se recomienda realizar la transición del prototipo estático a un sistema completamente operativo en la siguiente fase del proyecto. Esto implica sustituir los disparadores temporales de MessageBox por la invocación de funciones controladoras reales que capturen las entradas de los formularios en Tkinter y ejecuten las correspondientes sentencias SQL (*Create, Read, Update, Delete*) directamente sobre el archivo de persistencia local clinica.db.
- ***Optimización Algorítmica de Búsqueda y Ordenamiento:*** Con el fin de prever un crecimiento en el volumen de datos de la clínica, se sugiere sustituir las búsquedas lineales secuenciales actuales por la implementación de algoritmos de búsqueda binaria sobre campos clave indexados como la Cedula. Para ello, se debe asegurar que los arreglos o listas dinámicas en memoria se encuentren previamente ordenados mediante la codificación de algoritmos avanzados como *Merge Sort* o *Quick Sort* clasificados por especialidad, fecha y bloques horarios.
- ***Migración de Arquitectura de Datos y Escalabilidad:*** Ante un eventual despliegue en un entorno de producción real, se recomienda planificar la migración del motor de base de datos embebido SQLite hacia un sistema de gestión de bases de datos relacionales robusto y distribuido (como PostgreSQL o MySQL). Esto permitirá mitigar las restricciones operativas locales, soportar accesos concurrentes múltiples por parte del personal administrativo y médico, y garantizar la escalabilidad del sistema de información.
- ***Fortalecimiento de la Seguridad y Validación de Datos:*** Se sugiere incorporar una capa robusta de validación de datos en el *backend* antes de realizar la persistencia en la base de datos. Esto debe incluir la verificación de expresiones regulares para campos críticos (como cédulas de identidad ecuatorianas válidas, formatos de números telefónicos y rangos de edad permitidos), evitando la inyección de datos corruptos o vulnerabilidades de seguridad en el sistema de gestión.

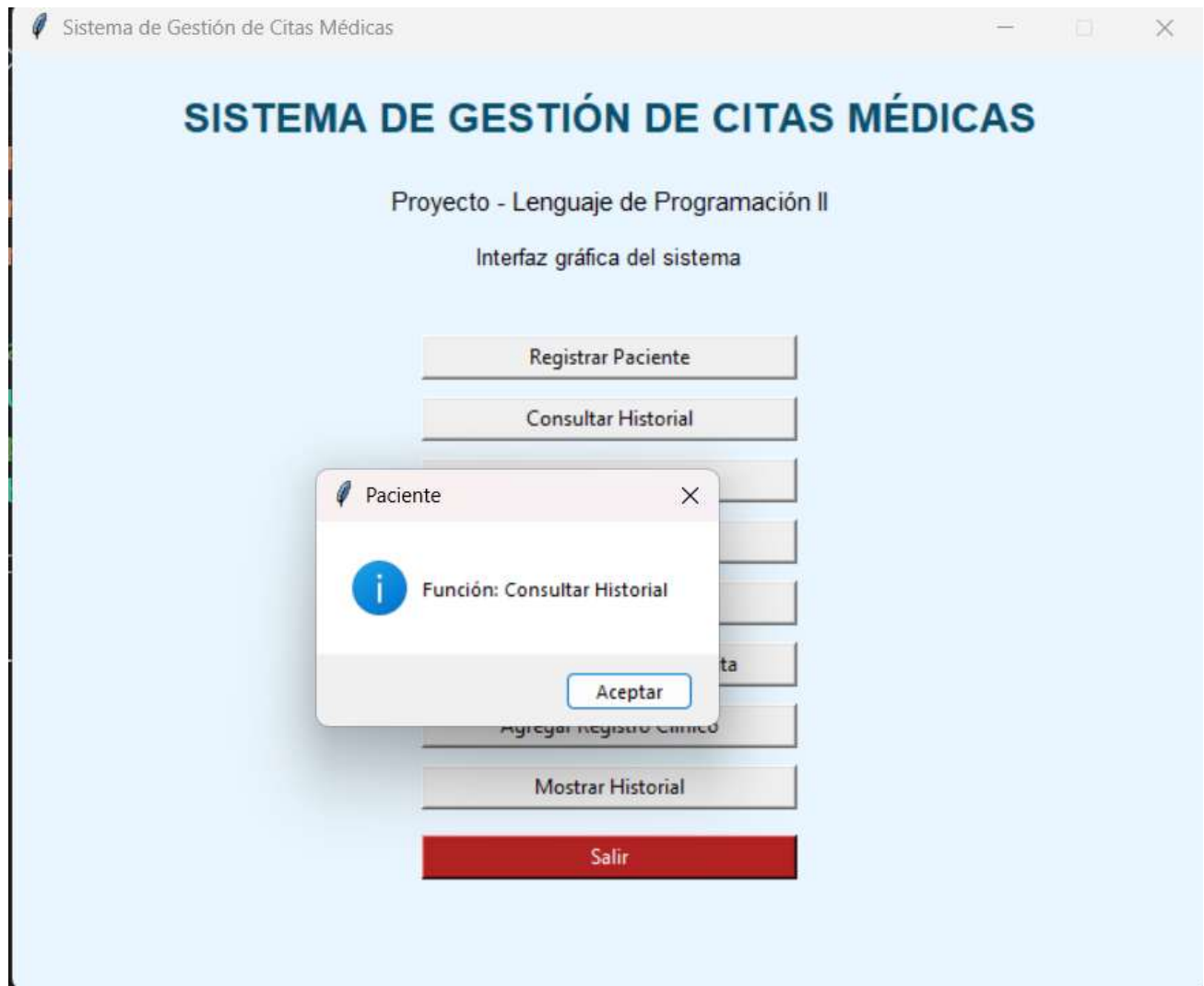
BIBLIOGRAFÍA

- Bel, W. (2020). Algoritmos y estructuras de datos en Python. Facultad CyT. p17. Ed Uader. <https://editorial.uader.edu.ar/sites/default/files/imagenes-sitio/colecciones/amalgama/algoritmos-y-estructuras-de-datos-en-phyton/algoritmos-y-estructuras-de-datos-en-python-digital.pdf>
- Valdivia, F. de L. P., Baca, H. A. H., & Solís, I. S. (2022). Estructuras de Datos y Algoritmos con Python. Iván Soria Solís. <https://books.google.com.pe/books?id=nGFzEAAAQBAJ&printsec=frontcover&hl=es#v=onepage&q&f=false>
- G, M. A. C. (2024b). Programación para Ingenieros: Python con Programación Orientada a Objetos. Mario Andrés Cuevas.
- Hernández, M., & Baquero, L. (2025). *Python con orientación a objetos y al análisis de datos*. Ediciones de la U. https://www.google.com.ec/books/edition/Python_con_orientaci%C3%B3n_a_objetos_y_al_a/AIVGEQAAQBAJ?hl=es-419&gbpv=1&dq=Python+con+Programaci%C3%B3n+Orientada+a+Objetos&pg=PA7&printsec=frontcover
- Bahit, E. (2022). Python Aplicado: 4a Edición. EBRC Publisher. https://www.google.com.ec/books/edition/Python_Aplicado/gBFyEAAAQBAJ?hl=es-419&gbpv=1&dq=Programaci%C3%B3n+Orientada+a+Objetos+en+Python&pg=PA267&printsec=frontcover
- Pressman, R. S., & Maxim, B. R. (2024). *Ingeniería de software: Un enfoque práctico* (9.^a ed.). McGraw-Hill Interamericana. <https://www.mheducation.es/ingenieria-de-software-un-enfoque-practico-9781456281434-es-mx>
- Sommerville, I. (2022). *Ingeniería de sistemas de software global* (11.^a ed.). Pearson Educación. <https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003254/9780133943030>

ANEXOS

Anexo A. Evidencias de la interfaz gráfica





SISTEMA DE GESTIÓN DE CITAS MÉDICAS

Proyecto - Lenguaje de Programación II

Interfaz gráfica del sistema

Registrar Paciente

Consultar Historial

Médico

i

Función: Registrar Médico

Aceptar

ita

Agregar Registro Clínico

Mostrar Historial

Salir

SISTEMA DE GESTIÓN DE CITAS MÉDICAS

Proyecto - Lenguaje de Programación II

Interfaz gráfica del sistema

Registrar Paciente

Consultar Historial

Médico

i

Función: Consultar Agenda

Aceptar

ita

Agregar Registro Clínico

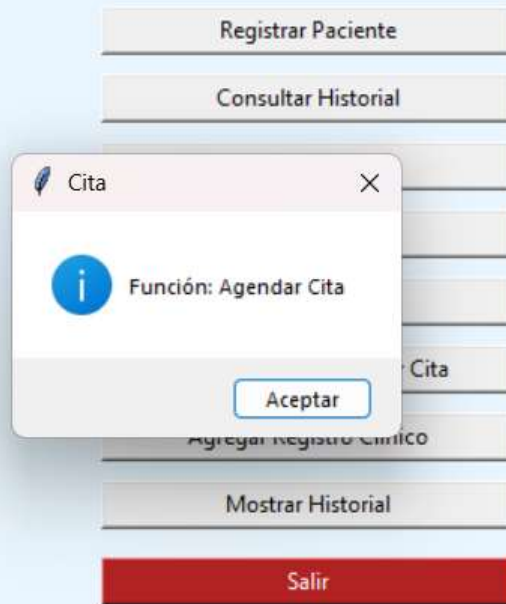
Mostrar Historial

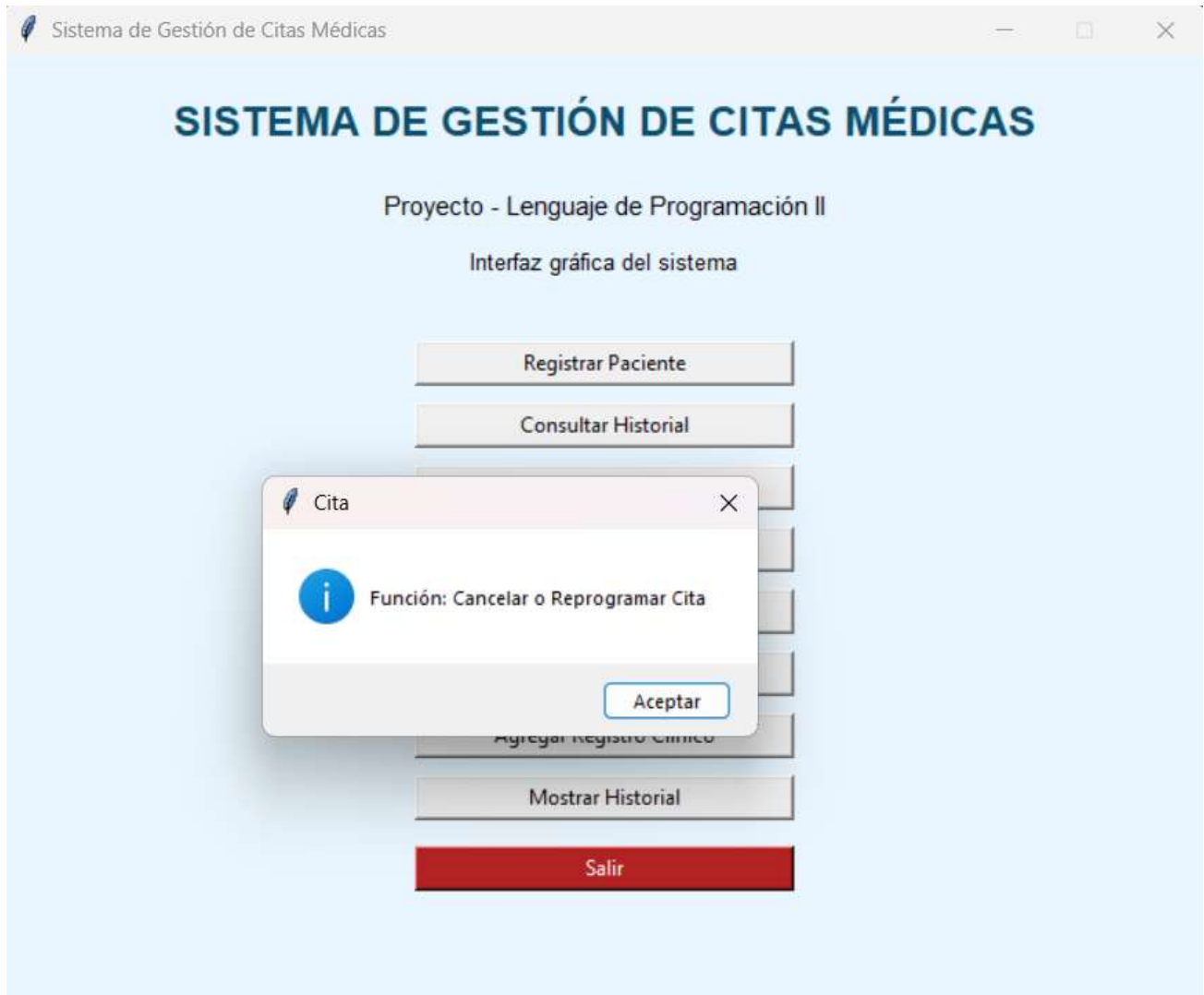
Salir

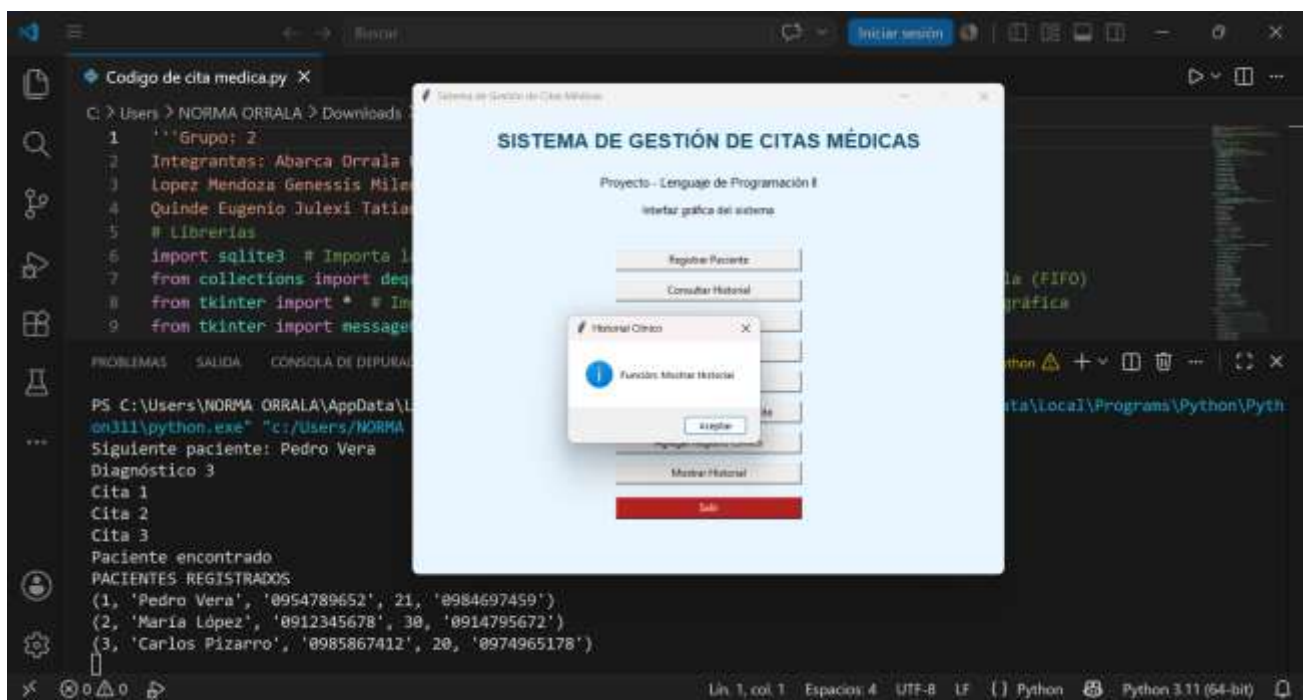
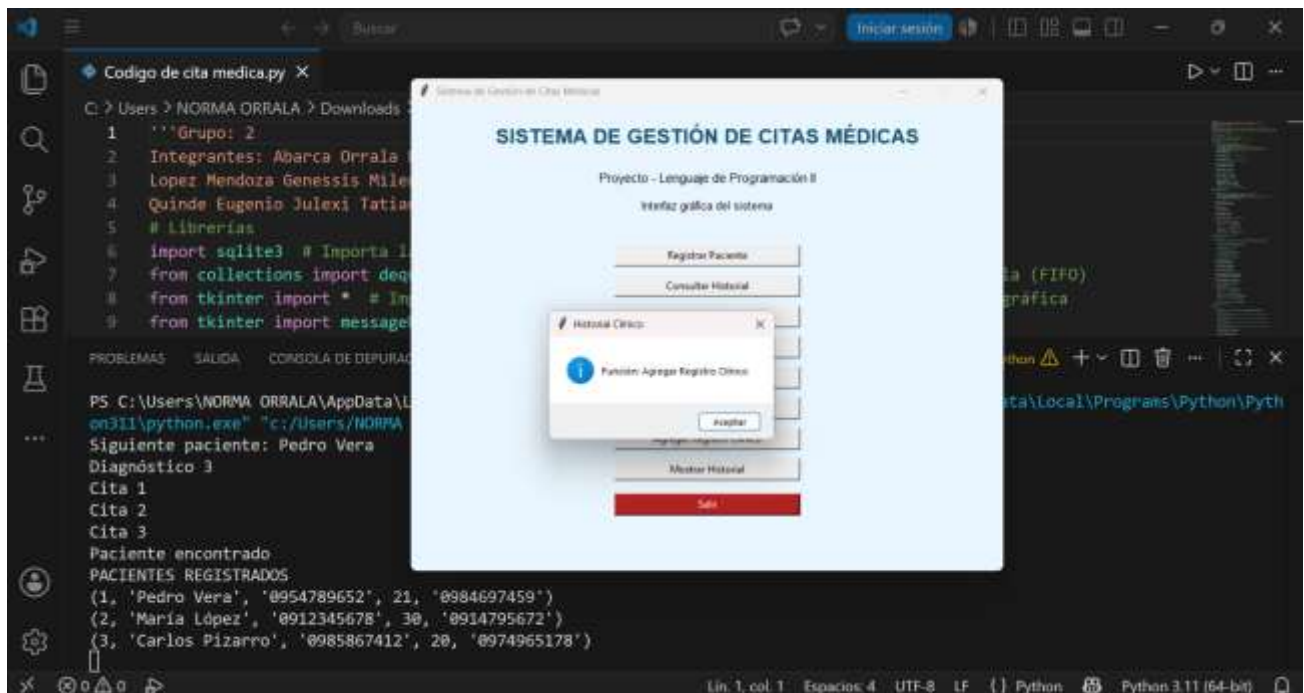
SISTEMA DE GESTIÓN DE CITAS MÉDICAS

Proyecto - Lenguaje de Programación II

Interfaz gráfica del sistema







Registrar Paciente

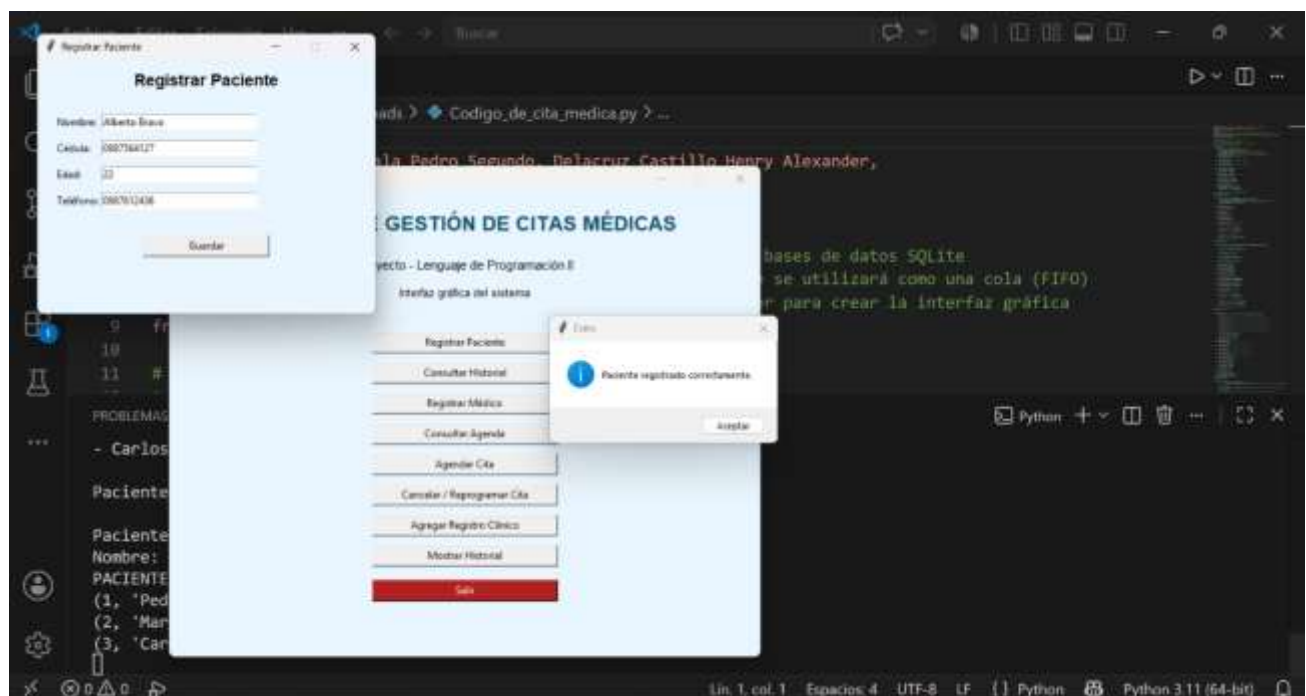
Registrar Paciente

Nombre:

Cédula:

Edad:

Teléfono:



Anexo B. Evidencias de la base de datos

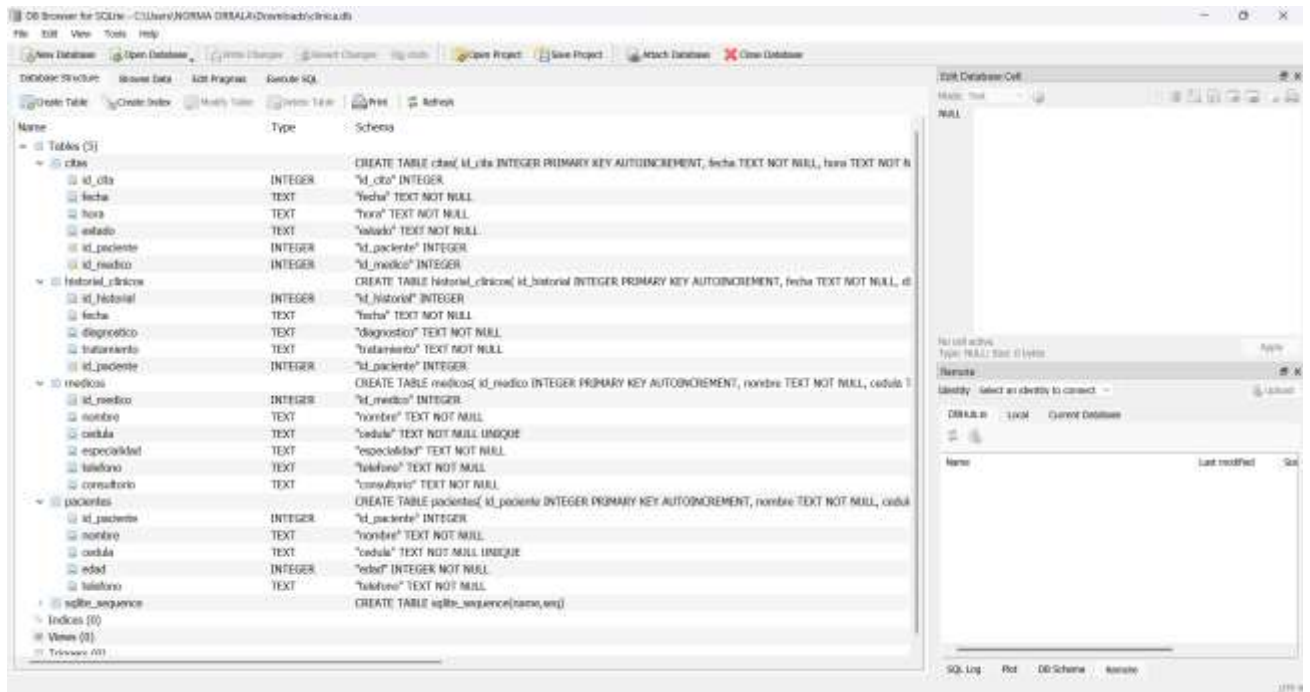


Table: **pacientes**

	<u>id_paciente</u>	nombre	cedula	edad	telefono
	Filter	Filter	Filter	Filter	Filter
1	1	Pedro Vera	0954789652	21	0984697459
2	2	María López	0912345678	30	0914795672
3	3	Carlos Pizarro	0985867412	20	0974965178

New Database Open Database Write Changes Revert Changes

Database Structure Browse Data Edit Pragmas

Table: **citas**

	<u>id_cita</u>	fecha	hora	estado	<u>id_paciente</u>	<u>id_medico</u>
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	20/07/2026	08:00	Pendiente	1	1
2	2	21/07/2026	09:00	Atendida	2	2
3	3	22/07/2026	10:30	Pendiente	3	3

Table: **medicos** Filter in any column

	<u>id_medico</u>	nombre	cedula	especialidad	telefono	consultorio
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	Carlos Mora	1100110011	Cardiología	0981111111	101
2	2	Ana Torres	1200124785	Pediatría	0982222222	102
3	3	Luis Gómez	1236543795	Dermatología	0983333333	103

Table: **historial_clinicos** Filter in any column

	<u>id_historial</u>	fecha	diagnostico	tratamiento	<u>id_paciente</u>
	Filter	Filter	Filter	Filter	Filter
1	1	20/07/2026	Gripe	Paracetamol	1
2	2	21/07/2026	Migraña	Ibuprofeno	2
3	3	22/07/2026	Alergia	Loratadina	3

DB Browser for SQLite - C:\Users\NORMA ORTIZ\AppData\Local\Programs\Microsoft VS Code\bin\sqlite.exe

File Edit View Tools Help

New Database Open Database Recent Databases Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

Table: **paciente** Filter in any column

	<u>id_paciente</u>	nombre	cedula	edad	telefono
	Filter	Filter	Filter	Filter	Filter
1	1	Medro Mora	0664700652	21	0984447448
2	2	María López	0512345678	35	0914795072
3	3	Cecilia Vizarro	0900867412	20	0974060179
4	4	Alberto Bravo	0607506127	22	0997012436

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100

1 of 4

SQLite: 1

SQL Log Plot DB Schema Run SQL

1 of 1

Editing row=1, column=1
(Type: Text / Numeric Size: 1 character(s))

Apply

Refresh

Identify: Select an identity to connect

Identify Local Current Database

Name Last modified Size

Anexo C. Evidencias de ejecución del programa

```
PROBLEMAS    SALIDA    CONSOLA DE DEPURACIÓN    TERMINAL    PUERTOS
```

```
=====
PILA (HISTORIAL CLÍNICO)
=====
Historial almacenado:
['Gripe', 'Migraña', 'Alergia']

Última atención registrada:
Alergia

Se elimina la última atención

Historial actualizado
['Gripe', 'Migraña']
```

```

=====
LISTA DE CITAS
=====
Citas registradas:
['20/07/2026 08:00', '21/07/2026 09:00', '22/07/2026 10:30']

Recorrido de la lista:
20/07/2026 08:00
21/07/2026 09:00
22/07/2026 10:30

=====
BÚSQUEDA SECUENCIAL
=====
Lista de pacientes registrados:
- Pedro
- María
- Carlos

Paciente a buscar: Carlos

Paciente encontrado correctamente.
Nombre: Carlos

PACIENTES REGISTRADOS
(1, 'Pedro Vera', '0954789652', 21, '0984697459')
(2, 'María López', '0912345678', 30, '0914795672')
(3, 'Carlos Pizarro', '0985867412', 20, '0974965178')

```

Enlace del repositorio: <https://github.com/abarcaorralapedrosegundo-crypto/PROYECTO.git>